Clasificación de Especies de Mariposas mediante Aprendizaje Profundo

Emiliano Camacho Ponce
Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Querétaro

I. INTRODUCCIÓN

La clasificación automática de especies a partir de imágenes es una de las aplicaciones más representativas de la visión por computadora en las ciencias biológicas. Los insectos, y en particular las mariposas, son indicadores ecológicos importantes. Su presencia y abundancia reflejan la salud de un ecosistema, por lo que su monitoreo es valioso para la conservación y la biodiversidad. Sin embargo, identificar una especie a simple vista requiere conocimiento experto, ya que muchas especies comparten patrones de color y forma muy similares.

En este trabajo se aborda el problema de clasificar imágenes de mariposas en 75 especies distintas. Se trata de un problema de clasificación multiclase de imágenes, para el cual las redes neuronales convolucionales (CNN) son la arquitectura de referencia, ya que aprenden de manera automática bordes, texturas, patrones de las alas, sin necesidad de diseñarlas manualmente.

El proyecto se desarrolla en dos etapas con el fin de compararlas bajo las mismas condiciones. Primero, una CNN construida y entrenada desde cero, y segundo, un modelo basado en transfer learning, que utiliza una red pre entrenada. Posteriormente se analizan los resultados en validación, se aplican técnicas de regularización y ajuste de hiperparámetros, y se documenta qué cambios mejoraron el desempeño y por qué. Toda la solución se implementó utilizando Python como lenguaje de programación base.

II. ETL

A. ¿Qué es ETL?

ETL es un acrónimo de Extracción, Transformación y Carga (Extract, Transform, Load), un proceso fundamental en ciencia de datos que consiste en obtener datos de una fuente, limpiarlos y transformarlos a un formato adecuado, y finalmente cargarlos para su uso en análisis o modelos de aprendizaje. En visión por computadora, este proceso incluye además verificar la integridad de los archivos de imagen y prepararlos (redimensionado, escalado, codificación de etiquetas) para que puedan ser consumidos por una red neuronal.

B. Datos

Se utilizó el dataset Butterfly Image Classification, de dominio público en Kaggle, que contiene imágenes de mariposas pertenecientes a 75 especies. El dataset consta de 9,285 imágenes en formato JPG, organizadas en dos carpetas: train (6,499 imágenes) y test (2,786 imágenes). La información de las etiquetas se proporciona en dos archivos CSV, descritos en la Tabla 1.

Archivos	Registros y Columnas	Descripción
Training_set.csv	6,499 filename, label	Nombre de archivo de cada imagen de la carpeta train y la especie de mariposa a la que pertenece.
Testing_set.csv	2,786 filename	Nombre de archivo de cada imagen de la carpeta test, sin etiqueta. El orden de los nombres corresponde al orden en que deben hacerse las predicciones

Tabla 1. Archivos del dataset

El dataset es de dominio público y se puede acceder y consultar a través del siguiente enlace: https://www.kaggle.com/datasets/phucthaiv02/butterfly-image-classification

C. Desarrollo

1. Extracción

Se descargó el dataset desde Kaggle y, utilizando la biblioteca pandas, se cargaron ambos archivos CSV en DataFrames. Las imágenes se leen posteriormente desde sus

carpetas a partir de los nombres de archivo indicados en cada CSV.

2. Transformación

Se comprobó que Training_set.csv y Testing_set.csv no contienen valores faltantes, que no existen nombres de archivo duplicados dentro de cada conjunto, y que todas las imágenes pudieron abrirse sin errores, por lo que no fue necesario eliminar ni imputar registros. Cabe señalar que los nombres de archivo siguen la numeración Image_1.jpg, Image_2.jpg, etc. de forma independiente en cada carpeta, por lo que un mismo nombre aparece tanto en train como en test y corresponde a imágenes distintas. Por esta razón, las imágenes siempre se leen indicando explícitamente la carpeta de origen.

En Training_set.csv se identificaron 75 clases. La distribución es relativamente balanceada ya que cada clase tiene entre 71 y 131 imágenes, con un promedio de 86.7 imágenes por clase. La clase más representada es MOURNING CLOAK (131 imágenes) y la menos representada es WOOD SATYR (71). Dado que el desbalance es leve, se consideró suficiente conservar la proporción de clases en las divisiones del dataset, sin aplicar técnicas de balanceo adicionales.

Las etiquetas son strings con el nombre de la especie, por lo que se convirtieron a índices enteros de 0 a 74, y se guardó el mapeo inverso para poder recuperar el nombre de la especie a partir de la predicción del modelo. Para el entrenamiento, la salida de la red usa una capa softmax de 75 neuronas.

El conjunto de test proporcionado por el dataset no incluye etiquetas, por lo que no es posible calcular métricas de desempeño con él. Por esta razón, el conjunto de 6,499 imágenes etiquetadas se dividió en tres partes: train con el 70%, validation con el 15% y test con el 15%. El conjunto de train se usa para ajustar los parámetros del modelo, el de validation para monitorear la generalización y ajustar hiperparámetros durante el entrenamiento, y el de test se reserva exclusivamente para la evaluación final. Las 2,786 imágenes de la carpeta test original, al no tener etiqueta, se utilizan únicamente para realizar predicciones de demostración con el modelo final.

Todas las imágenes se redimensionaron a una resolución fija de 256 × 256 píxeles, necesaria para que la red reciba entradas del mismo tamaño, y sus valores de píxel se normalizaron al rango [0, 1] dividiendo entre 255 (para el modelo con transfer learning se usa el preprocesamiento propio de la red pre entrenada). En esta etapa no se aplica aumento de datos; este se incorporará más adelante, únicamente sobre el conjunto de train, como técnica de regularización.

3. Carga

Los conjuntos de train, validation y test resultantes se cargan mediante un pipeline de datos que lee las imágenes por lotes (batches), aplica el preprocesamiento descrito y las entrega al modelo. Las listas de archivos con sus etiquetas de cada división se guardan en archivos CSV para garantizar que ambos modelos se entrenen y evalúen exactamente con los mismos datos y puedan compararse bajo las mismas condiciones.

III. MODELO 1: CNN desde cero (base)

1. ¿Qué es una red neuronal convolucional?

Una red neuronal convolucional (CNN) es una arquitectura de aprendizaje profundo diseñada para procesar datos con estructura espacial, como las imágenes. En lugar de conectar cada píxel con cada neurona, como una red perceptrón multicapa, utiliza filtros pequeños que se deslizan sobre la imagen y detectan patrones locales. Las primeras capas aprenden patrones simples, como bordes y colores, y las capas profundas los combinan en patrones más complejos, como las manchas y venas características de las alas de cada especie.

La red se construye a partir de los siguientes tipos de capas:

• **Convolución (Conv2D):** aplica un conjunto de filtros de 3 × 3 y produce un mapa de características por filtro. Sus pesos se aprenden durante el entrenamiento.

• **Batch Normalization:** normaliza las activaciones de cada lote a media 0 y varianza 1. Estabiliza y acelera el entrenamiento.

• **ReLU:** función de activación $f(x) = \max(0, x)$, que introduce no linealidad en la red.

• **Max Pooling:** reduce a la mitad el alto y ancho del mapa conservando el valor máximo de cada ventana de 2 × 2. Disminuye el costo de cómputo y da tolerancia a pequeños desplazamientos.

• **Global Average Pooling:** promedia cada mapa de

características final en un solo valor, convirtiendo el tensor en un vector.

• **Softmax:** capa de salida con una neurona por clase. La función softmax convierte sus salidas en una distribución de probabilidad sobre las 75 especies.

2. Arquitectura

La red se diseñó con cuatro bloques convolucionales, cada uno formado por Conv2D → BatchNormalization → ReLU → MaxPooling. El número de filtros se duplica en cada bloque (32, 64, 128 y 256) mientras la resolución se reduce a la mitad, de modo que la red gana capacidad para representar patrones complejos conforme la imagen se hace más pequeña. Al final se usa Global Average Pooling en lugar de aplanar el tensor (Flatten), lo que reduce el número de parámetros de la capa de salida de millones a unos miles y disminuye el riesgo de overfitting.

Capa	Tamaño de salida	Parámetros
Entrada	256 × 256 × 3	0
Bloque 1: Conv2D (32) + BN + ReLU + MaxPool	128 × 128 × 32	992
Bloque 2: Conv2D (64) + BN + ReLU + MaxPool	64 × 64 × 64	18,688
Bloque 3: Conv2D (128) + BN + ReLU + MaxPool	32 × 32 × 128	74,240
Bloque 4: Conv2D (256) + BN + ReLU + MaxPool	16 × 16 × 256	295,936
Global Average Pooling	256	0
Dense (softmax)	75	19,275
Total		**409,131**

Tabla 2. Arquitectura de la CNN desde cero

3. Entrenamiento

El entrenamiento ajusta los pesos de la red para minimizar la función de pérdida de entropía cruzada categórica (sparse categorical cross-entropy), que penaliza al modelo cuando asigna baja probabilidad a la clase correcta. Los pesos se actualizan con el optimizador Adam, una variante del descenso de gradiente que adapta la tasa de aprendizaje de cada parámetro.

Hiperparámetro	Valor
Optimizador	Adam
Tasa de aprendizaje	0.001
Función de pérdida	Sparse categorical cross-entropy
Tamaño de lote	32
Épocas	30
Resolución de entrada	256 × 256

Tabla 3. Hiperparámetros del modelo base

Esta primera versión se entrenó intencionalmente sin técnicas de regularización (aumento de datos, dropout o early stopping), con el fin de establecer una línea base y poder medir después el efecto de incluir estas mejoras por separado. Durante el entrenamiento se guardó el modelo de la época con mayor accuracy de validación, y ese modelo es el que se evaluó al final. El entrenamiento se realizó con una GPU NVIDIA RTX 4070 Ti usando TensorFlow 2.21.

IV. RESULTADOS DEL MODELO BASE

1. Métricas finales

Conjunto	Loss	Accuracy
Train	0.6172	81.53%
Validation	1.1788	68.72%
Test	1.2476	68.00%

Tabla 4. Métricas de la CNN desde cero (mejor época)

El modelo clasifica correctamente el 68.0% de las imágenes de test.. Los resultados de validation y test son prácticamente iguales, lo que indica que el conjunto de validación es representativo y que la selección de la mejor época no sobreajustó a ese conjunto.

2. Curvas de entrenamiento

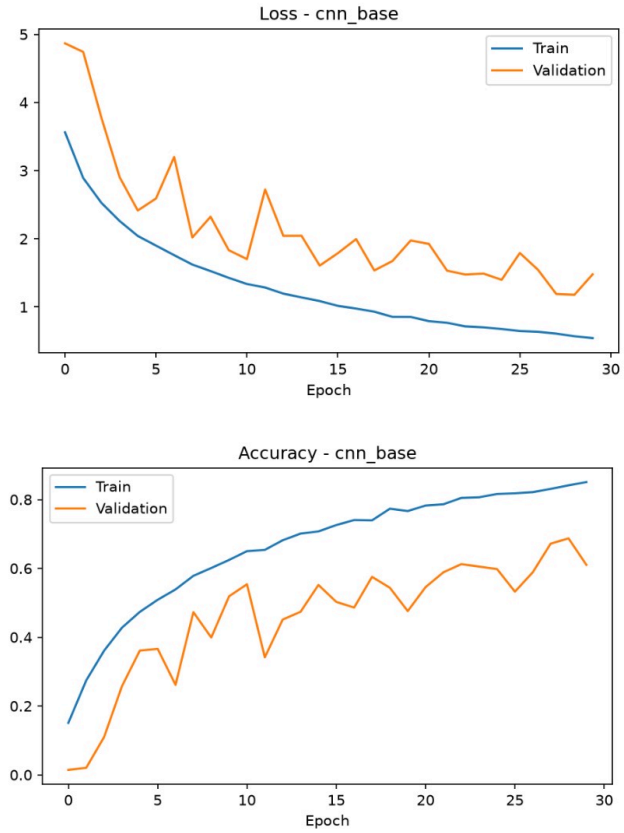


Figura 1. Loss y accuracy de train y validation a lo largo de las épocas (modelo base)

La curva de train desciende de forma suave y constante, mientras que la de validation baja con oscilaciones fuertes, con picos de pérdida en varias épocas. Desde las primeras épocas se abre una brecha entre ambas curvas que se mantiene durante todo el entrenamiento.

3. Análisis de sesgo y varianza

Overfitting: el accuracy de train (81.5%) supera al de validation (68.7%) por 12.8 puntos, y la pérdida de validation es casi el doble que la de train. El modelo memoriza detalles particulares de las imágenes de entrenamiento que no generalizan a imágenes nuevas. Esto era de esperarse, ya que cada especie cuenta con solo unas 60 imágenes de entrenamiento.

Sesgo moderado: el accuracy de train seguía subiendo al terminar las 30 épocas, por lo que el modelo todavía no había convergido y podría seguir aprendiendo con más entrenamiento.

Inestabilidad en validation: las oscilaciones de la curva de validation se deben a que la tasa de aprendizaje se mantiene fija durante todo el entrenamiento, lo que provoca saltos grandes en los pesos aun cerca de una buena solución.

Estos tres problemas guían la siguiente fase de este proyecto, donde se aplicarán técnicas de regularización para reducir la brecha entre train y validation, y se ajustará la tasa de aprendizaje para estabilizar el entrenamiento.

V. MEJORA DEL MODELO BASE

Con base en el diagnóstico hecho del modelo base, se aplicaron tres mejoras en una sola corrida. La arquitectura convolucional, los datos y el procedimiento de evaluación se mantuvieron iguales para que la comparación con el modelo base sea pareja.

1. Mejoras implementadas

Data augmentation: en cada época, cada imagen de entrenamiento se transforma de forma aleatoria antes de entrar a la red: giro horizontal, rotación de hasta ±36°, zoom de hasta ±15% y variación de contraste de hasta ±20%. Así el modelo nunca ve dos veces exactamente la misma imagen, lo que equivale a ampliar artificialmente el conjunto de entrenamiento.

Las transformaciones se implementaron como capas dentro del modelo, por lo que solo son aplicadas durante el entrenamiento; validación, prueba y predicción usan la imagen original. Las transformaciones elegidas no alteran la especie, ya que una mariposa sigue siendo la misma especie si se refleja, se gira ligeramente o cambia la iluminación.

Dropout: se agregó una capa de Dropout con tasa de 0.3 entre el Global Average Pooling y la capa de salida. En cada paso de entrenamiento desactiva al azar el 30% de las 256 características, lo que obliga a la red a no depender de unas pocas características específicas y a distribuir la información entre todas.

Learning rate adaptativo y más épocas: se usó

ReduceLROnPlateau, que reduce el learning rate a la mitad cada vez que la pérdida de validación pasa 5 épocas sin mejorar, con un mínimo de 1×10^{-6}. Además, el máximo de épocas se subió de 30 a 100 y se agregó early stopping con paciencia de 15 épocas para detener el entrenamiento si la pérdida de validación deja de mejorar.

Hiperparámetro	Base	Mejorado
Aumento de datos	No	Flip, rotación, zoom, contraste
Dropout	No	0.3
Tasa de aprendizaje	0.001 fija	0.001 inicial, reducción ×0.5 en meseta
Épocas máximas	30	100
Early stopping	No	Paciencia de 15 épocas

Tabla 5. Cambios entre el modelo base y el mejorado

2. Resultados

El entrenamiento usó las 100 épocas completas porque la pérdida de validación siguió mejorando lentamente hasta la época 99, por lo que el early stopping no se activó. La tasa de aprendizaje se redujo cinco veces, de 1×10^{-3} a 3.125×10^{-5}.

Conjunto	Loss	Accuracy mejorado	Accuracy base	Difer.
Train	0.2982	92.55%	81.53%	+11.0 pp
Validation	0.6877	83.18%	68.72%	+14.5 pp
Test	0.8058	79.69%	68.00%	+11.7 pp

Tabla 6. Métricas del modelo mejorado comparadas con el modelo base

El accuracy de test subió de 68.0% a 79.7%, una mejora de 11.7 puntos porcentuales, y la pérdida de test bajó de 1.25 a 0.81.

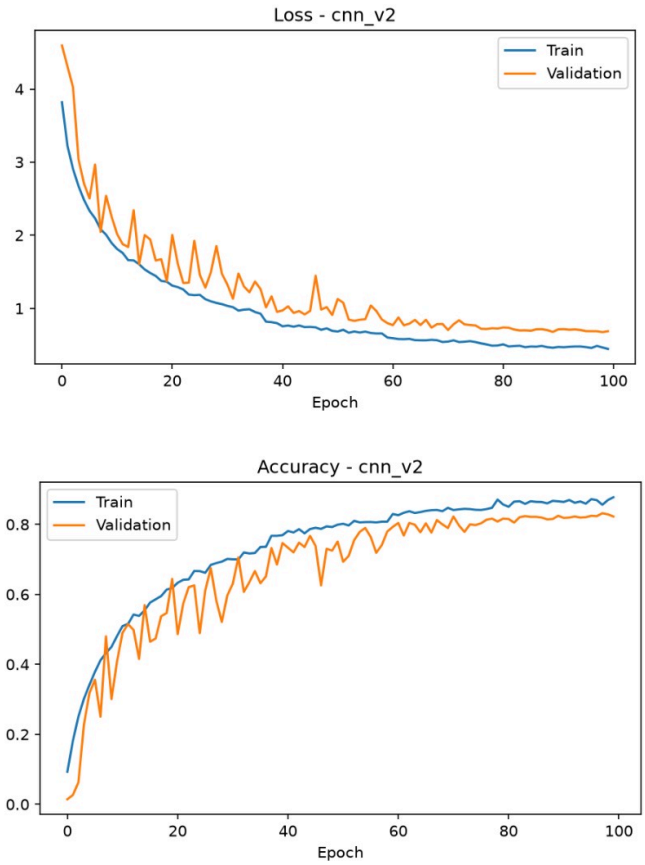


Figura 2. Loss y accuracy de train y validation a lo largo de las épocas (modelo mejorado)

3. Análisis de sesgo y varianza

El modelo mejorado redujo tanto el sesgo como la varianza. Alcanza un accuracy mayor en train (92.6%) y una brecha menor con validation. Aun así, persiste una brecha de 9.4 puntos, lo que indica que la varianza sigue siendo el principal problema del modelo.

El accuracy de test (79.7%) quedó 3.5 puntos por debajo del de validation (83.2%), una diferencia mayor que en el modelo base. Parte de esta diferencia proviene de que la mejor época se eligió con base en el desempeño en validation, lo que hace ese resultado ligeramente optimista.

El principal límite restante es la cantidad de datos, ya que con unas 60 imágenes por especie, una red entrenada desde cero debe aprender por sí sola tanto los patrones visuales generales (bordes, texturas, formas) como los rasgos específicos de cada especie. Este es justamente el problema que aborda el transfer learning, presentado en la siguiente sección.

VI. MODELO 2: TRANSFER LEARNING CON EFFICIENTNETB0

1. ¿Qué es transfer learning?

El transfer learning consiste en reutilizar una red neuronal previamente entrenada en un problema grande para resolver un problema nuevo con menos datos. Una CNN entrenada en ImageNet, un conjunto de más de un millón de imágenes de 1,000 categorías, ya aprendió en sus capas convolucionales a detectar bordes, texturas, formas y partes de objetos. Estos patrones visuales son útiles para casi cualquier problema de imágenes, por lo que solo es necesario reemplazar la capa de salida y ajustar la red a las 75 especies de mariposas.

Esta técnica ataca directamente el principal límite identificado en el modelo desde cero que no tiene datos suficientes para aprender desde cero tanto los patrones visuales como los rasgos de cada especie. Con transfer learning, los patrones generales ya vienen aprendidos y los datos disponibles se usan solo para aprender lo específico del problema.

2. Arquitectura

Se eligió EfficientNetB0 como red base. Es la versión más pequeña de la familia EfficientNet, que escala de forma balanceada la profundidad, el ancho y la resolución de la red, y logra un accuracy alto en ImageNet con relativamente pocos parámetros. Su base convolucional tiene 237 capas.

Sobre la red base, sin su capa de clasificación original, se agregó la misma cabeza que en el modelo desde cero mejorado: Global Average Pooling, Dropout de 0.3 y una capa softmax de 75 neuronas. También se conservó el mismo aumento de datos, de modo que la diferencia de resultados se deba solo al uso de la red preentrenada y no a otros cambios.

El EfficientNet de Tensorflow incluye su propia capa de normalización y espera valores de píxel en el rango de 0 a 255. Como el pipeline de datos entrega las imágenes normalizadas entre 0 y 1, se agregó una capa de reescalado (×255) antes de la red base, sin modificar el proceso de ETL.

3. Entrenamiento

El entrenamiento se dividió en dos fases:

Fase de extracción de características: se congelaron todos los pesos de la red base y solo se entrenó la cabeza nueva (96,075 parámetros), con tasa de aprendizaje de 1×10^{-3}. Si se entrenara toda la red desde el inicio, los gradientes grandes que produce una capa de salida con pesos aleatorios destruirían los patrones que la red base aprendió en ImageNet.

Fase de fine-tuning: una vez que la cabeza aprendió a clasificar, se descongelaron las últimas 50 capas de la red base (2.6 millones de parámetros entrenables) y se reentrenaron con una tasa de aprendizaje 100 veces menor, 1×10^{-5}. Las capas finales de la red son las que representan patrones más complejos y específicos de ImageNet; al ajustarlas, la red adapta esos patrones a los rasgos que distinguen a las mariposas. La tasa de aprendizaje baja permite hacer cambios pequeños sin perder lo aprendido.

Durante el fine-tuning, las capas de Batch Normalization de la red base se mantuvieron congeladas.

Hiperparámetro	Fase 1	Fase 2
Capas entrenables de la base	Ninguna	Últimas 50 de 237
Parámetros entrenables	96,075	2,604,763
Tasa de aprendizaje inicial	1×10^{-3}	1×10^{-5}
Épocas máximas	20	40
Épocas ejecutadas	20	18 (early stopping)

Tabla 7. Configuración de las dos fases de entrenamiento

En ambas fases se usaron ReduceLROnPlateau (reducción ×0.5 tras 3 épocas sin mejora) y early stopping (paciencia de 8 épocas). En la fase 2, el modelo solo se guardaba si superaba el mejor accuracy de validación de la fase 1.

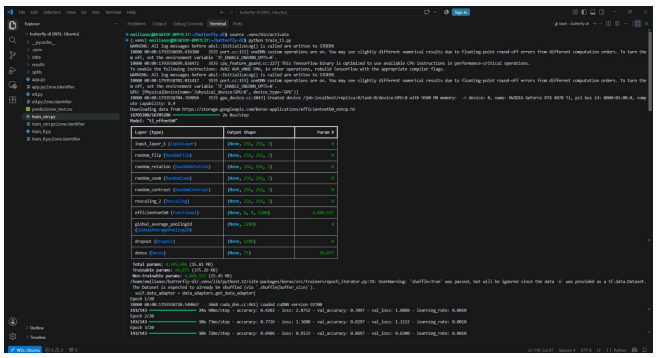


Figura 3. Evidencia de entrenamiento

4. Resultados

Conjunto	Loss	Accuracy
Train	0.0489	98.68%
Validation	0.2260	94.05%
Test	0.3238	91.69%

Tabla 8. Métricas del modelo con transfer learning (mejor época)

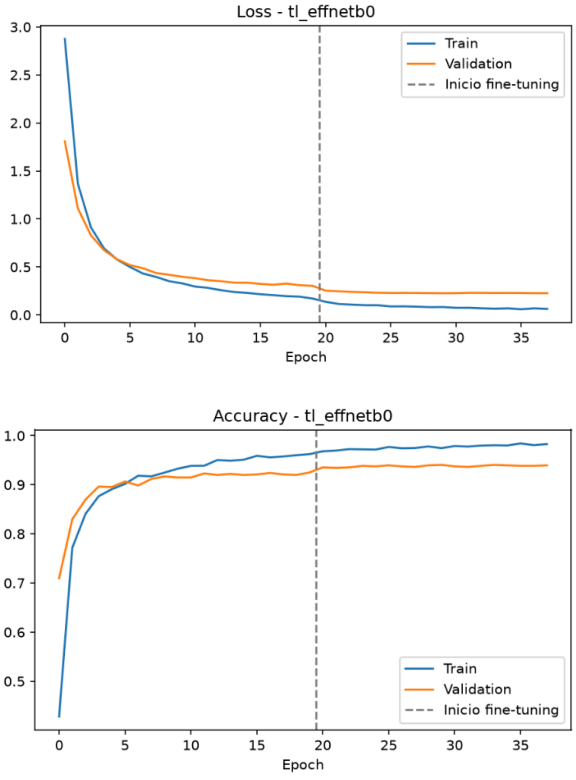


Figura 4. Loss y accuracy de train y validation (transfer learning).

Desde la primera época, con la red base congelada, el modelo ya alcanza 71% de accuracy de validación, superior al resultado final de la línea base desde cero (68.7%). Esto muestra que las características aprendidas en ImageNet son útiles para distinguir mariposas aun sin ajustarlas. Al final de la fase 1, la validación llega a cerca de 92%.

Al iniciar el fine-tuning, la pérdida de validación baja de forma visible y el accuracy de validación sube a cerca de 94%, donde se estabiliza. El early stopping detuvo la fase 2 tras 18 épocas, al no observar mejora en la pérdida de validación durante 8 épocas.

A diferencia de los modelos desde cero, las curvas de validación son suaves desde el inicio. La red base ya parte de pesos estables, por lo que el entrenamiento no presenta saltos grandes.

5. Análisis de sesgo y varianza

Sesgo bajo: con 98.7% de accuracy en train, el modelo tiene capacidad suficiente para representar las 75 especies.

Varianza moderada: la brecha entre train y validation es de 4.6 puntos, menos de la mitad que en el modelo desde cero mejorado (9.4). Al reutilizar patrones aprendidos con más de un millón de imágenes, el modelo depende menos de los detalles particulares del conjunto de entrenamiento.

El accuracy de test (91.7%) quedó 2.4 puntos por debajo del de validation (94.1%), una diferencia de la misma magnitud que en el modelo anterior y explicada por las mismas razones de la selección de la mejor época con el conjunto de validación y la variación de muestreo en conjuntos de 975 imágenes.

VII. COMPARACIÓN DE MODELOS

Modelo	Parámetros entrenables	Accuracy train	Accuracy val	Accuracy test	Loss test
CNN desde cero (línea base)	409,131	81.53%	68.72%	68.00%	1.2476
CNN desde cero mejorada	409,131	92.55%	83.18%	79.69%	0.8058
EfficientNetB0 con transfer learning	2,604,763 (fine-tuning)	98.68%	94.05%	91.69%	0.3238

Tabla 9. Comparación de los tres modelos

El modelo con transfer learning es el mejor en todas las métricas. Su accuracy de test (91.7%) supera en 12.0 puntos a la CNN desde cero mejorada y en 23.7 puntos al modelo

base.

Las mejoras de regularización aplicadas a la CNN desde cero aportaron 11.7 puntos de accuracy en test, pero no pudieron compensar la falta de datos. El transfer learning aportó una mejora similar adicional porque resuelve la causa de fondo de que la red ya no necesita aprender a ver con solo 60 imágenes por especie, sino sólo a distinguir las especies entre sí. Además, convergió en 38 épocas en total, contra 100 de la CNN desde cero mejorada.

Por estas razones, EfficientNetB0 con transfer learning se selecciona como modelo final para realizar las predicciones.

VIII. PREDICCIONES CON INTERFAZ GRÁFICA

1. Implementación

Para probar el modelo final se desarrolló una interfaz gráfica web con la biblioteca Gradio, que permite construir interfaces para modelos de machine learning desde un solo archivo de Python. La aplicación carga el modelo EfficientNetB0 guardado durante el entrenamiento y se ejecuta de forma local en el navegador.

Antes de cada predicción, la imagen recibe el mismo preprocesamiento usado en el entrenamiento, se redimensiona a 256 × 256 píxeles y sus valores se escalan al rango [0, 1].

La interfaz se organiza en tres pestañas:

• **Subir imagen:** el usuario carga cualquier fotografía de una mariposa y la interfaz muestra las cinco especies más probables con su probabilidad. Incluye seis imágenes de ejemplo del conjunto de test para probarla con un clic.

• **Muestra del set de test:** selecciona ocho imágenes al azar de las 2,786 imágenes sin etiqueta de la carpeta test y muestra cada una con la especie predicha y su nivel de confianza.

• **Set de test completo:** clasifica las 2,786 imágenes en el orden de Testing_set.csv y genera un archivo CSV descargable con el nombre de archivo, la especie predicha y la confianza de cada imagen.

2. Resultados de las predicciones

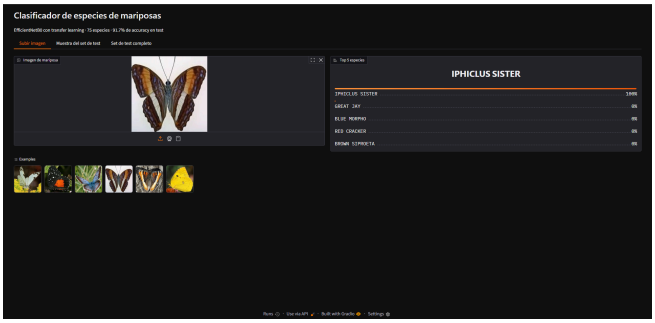


Figura 5. Predicción de una imagen individual en la pestaña Subir imagen

En la Figura 5, el modelo clasifica la imagen como IPHICLUS SISTER con una probabilidad cercana al 100%, mientras que las otras cuatro especies del top 5 tienen probabilidades cercanas a 0%. Esto indica que el modelo no tiene dudas entre especies para esta imagen.

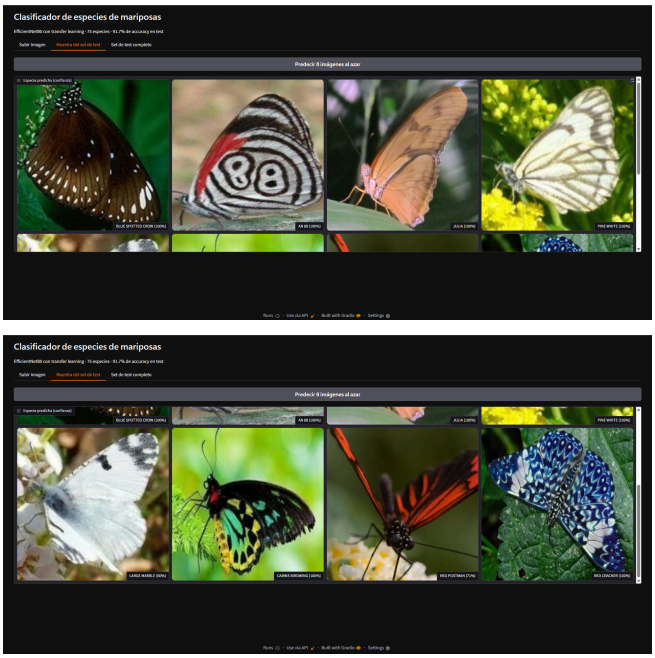


Figura 6. Predicciones sobre una muestra aleatoria de ocho imágenes del conjunto de test

En seis de las ocho imágenes el modelo predice con una confianza cercana al 100%. Las fotografías incluyen fondos variados y mariposas en distintas posiciones, con las alas abiertas o cerradas, lo que muestra que el modelo se enfoca en los patrones de las alas y no en el fondo.

Dos de las ocho imágenes tienen una confianza menor (60% y 71%). La confianza puede usarse como indicador para marcar las predicciones que conviene revisar manualmente.

IX. ANEXOS

https://github.com/Emiliano1410/butterflies_dl

X. REFERENCIAS

[1] phucthaiv02, "Butterfly Image Classification," Kaggle. [En línea]. Disponible en: https://www.kaggle.com/datasets/phucthaiv02/butterfly-image-classification

[2] IBM, "What is ETL (Extract, Transform, Load)?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/etl

[3] IBM, "What are convolutional neural networks?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/convolutional-neural-networks

[4] IBM, "What is overfitting?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/overfitting

[5] IBM, "What is regularization?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/regularization

[6] IBM, "What is data augmentation?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/data-augmentation

[7] IBM, "What is transfer learning?" IBM.com. [En línea]. Disponible en: https://www.ibm.com/think/topics/transfer-learning

[8] Wikipedia, "ImageNet." [En línea]. Disponible en: https://en.wikipedia.org/wiki/ImageNet

[9] Keras, "BatchNormalization layer." Keras documentation. [En línea]. Disponible en: https://keras.io/api/layers/normalization_layers/batch_normalization/

[10] Keras, "GlobalAveragePooling2D layer." Keras documentation. [En línea]. Disponible en: https://keras.io/api/layers/pooling_layers/global_average_pooling2d/

[11] Keras, "ReduceLROnPlateau." Keras documentation. [En línea]. Disponible en: https://keras.io/api/callbacks/reduce_lr_on_plateau/

[12] Keras, "EfficientNet B0 to B7." Keras documentation. [En línea]. Disponible en: https://keras.io/api/applications/efficientnet/

[13] Keras, "Transfer learning & fine-tuning." Keras documentation. [En línea]. Disponible en: https://keras.io/guides/transfer_learning/

[14] Keras, "Image classification via fine-tuning with EfficientNet." Keras documentation. [En línea]. Disponible en: https://keras.io/examples/vision/image_classification_efficientnet_fine_tuning/

[15] TensorFlow, "Data augmentation." TensorFlow Core. [En línea]. Disponible en: https://www.tensorflow.org/tutorials/images/data_augmentation

[16] TensorFlow, "Install TensorFlow with pip." [En línea]. Disponible en: https://www.tensorflow.org/install/pip

[17] scikit-learn, "train_test_split." scikit-learn documentation. [En línea]. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[18] Gradio, "Blocks." Gradio documentation. [En línea]. Disponible en: https://gradio.app/docs/gradio/blocks